

ЛР4 (3 пары / 6 часов)

«Сохранение задач в JSON-файл. Загрузка при старте. Папка data»

Что было до этого

У нас уже есть:

- `TaskTracker.Core`
 - `TaskItem` , `TaskStatus`
 - `TaskService` (Add/GetAll/ChangeStatus/Delete)
- `TaskTracker.App`
 - меню и работа с `TaskService`

Что делаем в ЛР4

Сейчас задачи **пропадают** после закрытия программы, потому что они “в памяти”.

В ЛР4 мы добавляем **хранение в файле JSON**, чтобы:

- добавил задачу → закрыл → открыл → задачи остались.

0) Итог ЛР4 (что должно работать)

- ✓ При старте программа **загружает** задачи из `tasks.json` (если он есть)
 - ✓ После добавления/изменения/удаления программа **сохраняет** задачи в `tasks.json`
 - ✓ Файл создаётся автоматически, папка `data` создаётся автоматически
 - ✓ Программа **не падает**, если файла ещё нет (пишет “нет данных, начинаем с пустого списка”)
-

1) Теория (важная и понятная)

1.1. Что такое JSON

JSON — это “текстовый формат данных”, который удобно хранить в файле.

Пример:

```
[
  { "Id": 1, "Title": "Купить тетрадь", "Status": 0 },
  { "Id": 2, "Title": "Сдать ЛР", "Status": 1 }
]
```

1.2. Сериализация и десериализация

- **Сериализация:** объект/список → JSON текст → файл
- **Десериализация:** файл JSON → список объектов в программе

1.3. Почему делаем отдельный “модуль хранения”

Чтобы не мешать “меню” и “логику задач” с работой с файлами.

Вводим новый модуль:

- `TaskTracker.Storage` — всё про чтение/запись файлов

Это помогает:

- легче поддерживать
- проще проверять
- потом пригодится для мобильной версии/тестов

1.4. Важно про пути к файлам (почему “не там” создаётся)

Если просто написать `"data\\tasks.json"`, файл может появиться в папке:

`bin\\Debug\\net8.0\\data\\...`

Это нормально для учебного проекта, но чтобы было стабильно, мы сделаем путь через:

`ApplicationContext.BaseDirectory` — “папка, где находится запущенная программа”.

2) Практика. Пара 1: создаём проект Storage и класс JsonTaskStorage

Шаг 1. Открыть решение

1. Visual Studio → открыть `TaskTracker.sln`

Шаг 2. Добавить новый проект `TaskTracker.Storage`

1. Solution Explorer → правой кнопкой по Solution "TaskTracker"
2. **Add** → **New Project...**
3. В поиске напишите **Class Library**
4. Выберите **Class Library (C#)**
5. **Next**
6. Project name: `TaskTracker.Storage`
7. **Create**

Шаг 3. Удалить Class1.cs

1. В `TaskTracker.Storage` найдите `Class1.cs`
2. Right click → Delete

Шаг 4. Добавить ссылку Storage → Core

Storage будет работать с моделью `TaskItem`, значит ему нужен Core.

1. В `TaskTracker.Storage` правой кнопкой по **Dependencies**
 2. **Add Project Reference...**
 3. Поставить галочку **TaskTracker.Core**
 4. OK
-

Шаг 5. Создать папку Services и файл JsonTaskStorage.cs

1. Правой кнопкой по `TaskTracker.Storage`
2. **Add** → **New Folder** → назвать `Services`
3. Правой кнопкой по `Services`
4. **Add** → **Class...**
5. Имя: `JsonTaskStorage.cs` → Add

Шаг 6. Вставить код JsonTaskStorage

Откройте `JsonTaskStorage.cs` и вставьте полностью:

```
using System.Text.Json;
using TaskTracker.Core.Models;

namespace TaskTracker.Storage.Services;

public class JsonTaskStorage
{
    private readonly string _filePath;

    public JsonTaskStorage(string filePath)
    {
        _filePath = filePath;
    }

    public List<TaskItem> Load()
    {
        // Если файла нет – возвращаем пустой список (это НЕ
        // ошибка)
        if (!File.Exists(_filePath))
            return new List<TaskItem>();
    }
}
```

```

        try
        {
            var json = File.ReadAllText(_filePath);
            var tasks = JsonSerializer.Deserialize<List<TaskItem>>(json);
            return tasks ?? new List<TaskItem>();
        }
        catch
        {
            // Если файл повреждён – не падаем, начинаем с пустого списка
            return new List<TaskItem>();
        }
    }

    public void Save(List<TaskItem> tasks)
    {
        var dir = Path.GetDirectoryName(_filePath);

        // Если папки нет – создаём
        if (!string.IsNullOrEmpty(dir))
            Directory.CreateDirectory(dir);

        var json = JsonSerializer.Serialize(tasks, new JsonSerializerOptions
        {
            WriteIndented = true
        });

        File.WriteAllText(_filePath, json);
    }
}

```

Шаг 7. Сборка решения

1. **Build** → **Build Solution**
 2. Должно быть **Build succeeded**
-

Шаг 8. Коммит №1 (Storage модуль)

Git Changes → сообщение:

- `feat: add Storage project with JsonTaskStorage`

Commit All → Push.

3) Практика. Пара 2: подключаем хранение к приложению (Load при старте, Save после действий)

Шаг 9. Подключить ссылку App → Storage

1. В `TaskTracker.App` правой кнопкой по **Dependencies**
 2. **Add Project Reference...**
 3. Галочка **TaskTracker.Storage**
 4. ОК
-

Шаг 10. Обновить TaskService, чтобы он умел стартовать с готовым списком

Сейчас `TaskService` сам создаёт пустой список и `_nextId=1`.

Нам нужно уметь "загрузить задачи из файла" и продолжить Id правильно.

Откройте: `TaskTracker.Core → Services → TaskService.cs`

10.1. Найдите поля и замените на такие

Замените верх класса на:

```

using TaskTracker.Core.Models;

namespace TaskTracker.Core.Services;

public class TaskService
{
    private readonly List<TaskItem> _tasks;
    private int _nextId;

    public TaskService(List<TaskItem>? initialTasks = null)
    {
        _tasks = initialTasks ?? new List<TaskItem>();

        // следующий Id = максимальный Id + 1
        _nextId = _tasks.Count == 0 ? 1 : _tasks.Max(t => t.Id) + 1;
    }

    public TaskItem Add(string title)
    {
        if (string.IsNullOrEmpty(title))
            throw new ArgumentException("Название не может быть пустым.");

        var task = new TaskItem
        {
            Id = _nextId++,
            Title = title.Trim(),
            Status = TaskStatus.New
        };

        _tasks.Add(task);
        return task;
    }
}

```

```

public List<TaskItem> GetAll()
{
    return _tasks.ToList();
}

private TaskItem GetExisting(int id)
{
    var task = _tasks.FirstOrDefault(t => t.Id == id);
    if (task is null)
        throw new ArgumentException($"Задача с Id={id} не
найдена.");
    return task;
}

public TaskItem ChangeStatus(int id, TaskStatus newStatus)
{
    var task = GetExisting(id);
    task.Status = newStatus;
    return task;
}

public void Delete(int id)
{
    var task = GetExisting(id);
    _tasks.Remove(task);
}
}

```

ВАЖНО: если красным подсвечивается `Max`, `ToList`, `FirstOrDefault` — сверху файла добавьте:

```
using System.Linq;
```

Соберите проект: Build → Build Solution.

Шаг 11. Подключаем Load/Save в Program.cs

Откройте `TaskTracker.App → Program.cs`.

11.1. Добавьте using

Вверх файла добавьте:

```
using TaskTracker.Storage.Services;
```

(У вас уже есть `using TaskTracker.Core...`)

11.2. Создайте путь к файлу данных

Сразу после `using` и до `while(true)` вставьте:

```
// Путь к файлу данных рядом с приложением (в папке запуска)
var dataFilePath = Path.Combine(AppContext.BaseDirectory, "data", "tasks.json");

// Хранилище JSON
var storage = new JsonTaskStorage(dataFilePath);

// Загружаем задачи из файла
var loadedTasks = storage.Load();

// Создаём сервис с уже загруженными задачами
var service = new TaskService(loadedTasks);

Console.WriteLine($"Данные: {dataFilePath}");
Console.WriteLine($"Загружено задач: {loadedTasks.Count}");
```

Эти 2 строки "Данные/Загружено" полезны новичкам — видно где файл.

Шаг 12. После каждого изменения — сохраняем

Теперь важно: **после Add / ChangeStatus / Delete** делать

```
storage.Save(service.GetAll());
```

12.1. В пункте "Добавить" (после успешного Add)

Найдите где вы делаете:

```
var task = service.Add(title);
```

Сразу после успешного добавления вставьте:

```
storage.Save(service.GetAll());
```

12.2. В пункте "Изменить статус" (после успешного ChangeStatus)

После:

```
var updated = service.ChangeStatus(id, newStatus);
```

добавьте:

```
storage.Save(service.GetAll());
```

12.3. В пункте "Удалить" (после service.Delete)

После:

```
service.Delete(id);
```

добавьте:

```
storage.Save(service.GetAll());
```

Шаг 13. Запуск и проверка сохранения (главная проверка)

1. Нажмите **F5**
 2. Добавьте 2 задачи
 3. Нажмите 0 → выйти
 4. Запустите снова **F5**
 5. Нажмите 2 → список должен быть НЕ пустой (задачи сохранились)
-

Шаг 14. Найти файл `tasks.json` на компьютере (чтобы увидеть, что он реально есть)

Прямо в консоли у вас печатается путь "Данные: ...".

Скопируйте его мышкой → откройте проводник → вставьте путь.

Там должен быть файл:

- `data\tasks.json`

Откройте его — увидите JSON.

Шаг 15. Коммит №2 (Load/Save подключены)

Сообщение:

- `feat: load tasks on startup and save after changes`

Commit All → Push.

4) Практика. Пара 3: защита от проблем + отчёт ЛР4

Шаг 16. Улучшение для новичков: сообщение, если файл был поврежден

Сейчас `Load()` просто возвращает пустой список при ошибке.

Для методички полезно сказать "данные повреждены".

Сделаем супер-простой вариант: в `JsonTaskStorage.Load()` добавим печать предупреждения в консоль — но **только один раз**.

Самый простой и безопасный способ: в `catch` добавить:

```
Console.WriteLine("Предупреждение: файл данных повреждён или не читается. Начинаем с пустого списка.");  
return new List<TaskItem>();
```

Это можно сделать, но если вы хотите, чтобы Storage был “тихий”, можно не делать. Для новичков полезно.

Шаг 17. Создать отчёт `docs/LR4_Report.md`

Создайте файл `docs/LR4_Report.md` и вставьте:

```
# Отчёт ЛР4 – TaskTracker
```

```
## Что сделано
```

- Добавлен проект TaskTracker.Storage
- Реализован JsonTaskStorage (Load/Save) для файла tasks.json
- TaskService теперь может стартовать с загруженным списком
- После Add/ChangeStatus/Delete данные сохраняются в JSON
- При старте данные загружаются

```
## Где лежит файл данных
```

Путь выводится в консоли при запуске: data/tasks.json (в папке запуска приложения)

```
## Как проверить
```

- 1) Запустить программу (F5)
- 2) Добавить 2 задачи
- 3) Выйти (0)
- 4) Запустить снова
- 5) Показать список (2) — задачи должны остаться

Шаг 18. Коммит №3 (отчёт + мелкие улучшения)

Сообщение:

- `docs: add LR4 report`

Commit All → Push.

5) Что сдавать (чек-лист)

Студент сдаёт:

✓ ссылку на репозиторий

✓ `docs/LR4_Report.md`

✓ скрин/видео 30–60 сек:

- запуск → видно "Загружено задач: ..."
 - добавили задачу
 - закрыли и запустили снова
 - список сохранился
-

6) Частые ошибки и решения (важно для новичков)

Ошибка 1: "Файл не находится / не вижу data/tasks.json"

Потому что файл создаётся рядом с exe:

- смотрите путь, который печатается в консоли ("Данные: ...")

Ошибка 2: "После загрузки Id снова с 1"

Значит не посчитали `_nextId` от максимального Id.

Проверьте, что в `TaskService` есть:

```
_nextId = _tasks.Count == 0 ? 1 : _tasks.Max(t => t.Id) + 1;
```

Ошибка 3: “Программа падает на чтении JSON”

Значит JSON поврежден. Нужно:

- `try/catch` в Load (у нас есть)
- и не падать, а начинать с пустого списка